

Challenge Clyvo — Olli Pet

Relatório Técnico de Evidências

DevOps Tools & Cloud Computing — Sprint 3

Este documento reúne, com prints e saídas reais de comandos, tudo o que foi implementado e validado para a entrega: aplicação rodando localmente via Docker Compose, deploy completo na Azure (ACR + ACI), CRUD funcionando ponta a ponta com prova de persistência no PostgreSQL, e o código publicado no GitHub. É material de apoio para a gravação do vídeo — não é o PDF oficial de entrega (que contém apenas nomes, RMs e links, conforme exigido no enunciado).

Repositório GitHub	https://github.com/HenriqueRodriguesV/java-ollipet-devops
API em produção (Azure)	http://ollipet-rm562917.eastus.azurecontainer.io:8080/swagger-ui.html
Resource Group	rg-rm562917-devops (região eastus)
Opção de entrega	ACR + ACI — containerização completa (App e Banco)
Banco de dados	PostgreSQL 16 (container, não é H2)
Vídeo de apresentação	(cole aqui o link do YouTube apos gravar)

1. Descrição da solução

A Olli Pet é uma clínica veterinária que hoje resolve agendamento, carteira de vacinação, triagem de urgência e acompanhamento de tratamento por conversas avulsas de WhatsApp e planilhas. A API é o backend do aplicativo mobile da clínica e concentra quatro regras de negócio centrais:

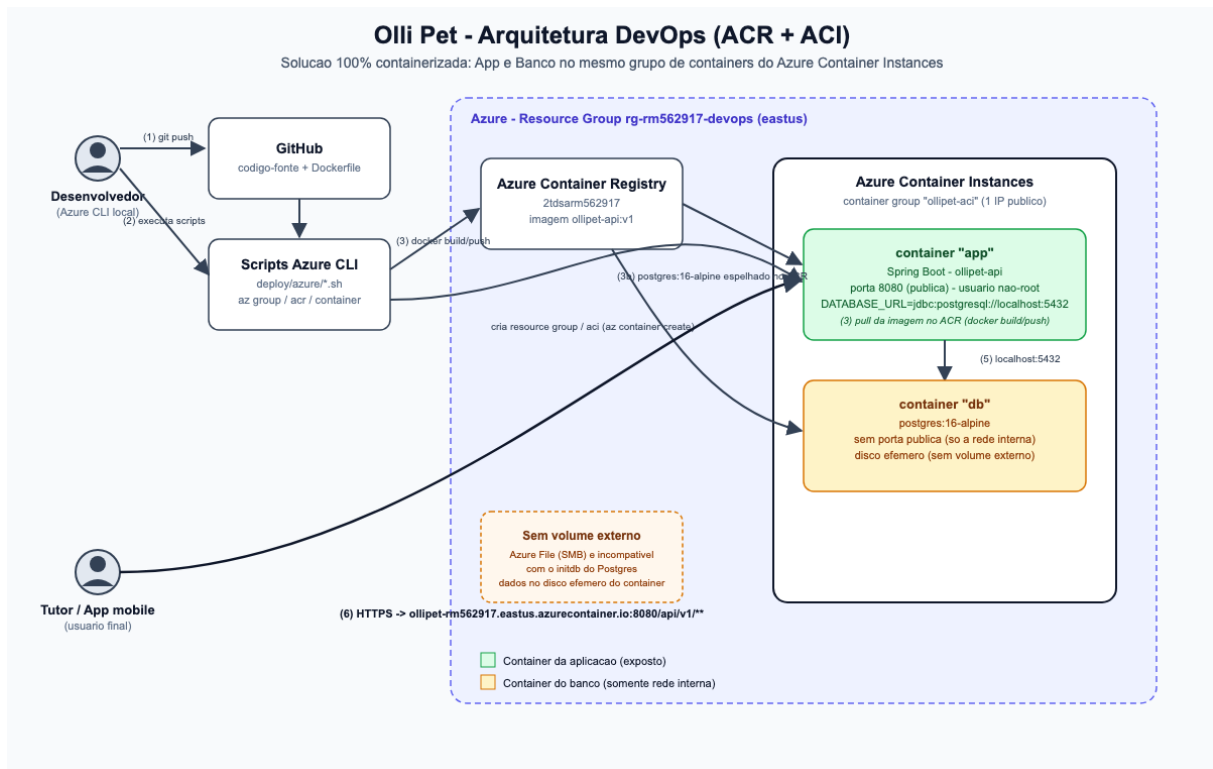
- **Triagem por questionário** — classifica a urgência de um pet a partir das respostas do tutor.
- **Agendamento de consulta** — controla a agenda dos veterinários, evita choque de horário.
- **Carteira de vacinação** — calcula sozinha o número da dose e a próxima aplicação.
- **Tratamento em casa** — transforma uma prescrição em doses individuais confirmáveis pelo tutor.

Benefícios para o negócio

- Menos falha humana na agenda: choques de horário e cancelamentos fora do prazo são recusados automaticamente.
- Triagem consistente 24h, sem depender de alguém da clínica responder no WhatsApp.
- Menos vacina atrasada: cálculo automático da próxima dose e lista de pendências.
- Adesão a tratamento mensurável, não mais "o tutor disse que deu o remédio".
- Rastreabilidade clínica: todo atendimento gera prontuário vinculado à consulta e ao profissional.

2. Diagrama da arquitetura

Solução 100% containerizada (App e Banco), seguindo a opção ACR + ACI: Azure Container Registry guarda a imagem, Azure Container Instances executa os dois containers no mesmo grupo (rede interna compartilhada — o banco nunca fica exposto na internet).



Fluxo completo: código no GitHub → scripts Azure CLI → build/push da imagem no ACR → container group no ACI → API pública consumida pelo app mobile / tutor.

3. Execução local (Docker Compose)

A aplicação sobe localmente com um único comando, já com o PostgreSQL e as migrations do Flyway aplicadas automaticamente:

```
$ docker compose up --build

Container ollipet-db   Started
Container ollipet-api  Started
```

Swagger UI rodando em <http://localhost:8090/swagger-ui.html>:

The screenshot displays the Swagger UI interface for an API. At the top, the Swagger logo is visible, along with a search bar containing "/api-docs" and an "Explore" button. Below this, the text "OpenAPI definition" is shown, followed by "v0" and "OAS 3.1". A "Servers" section indicates the server URL as "http://localhost:8090 - Generated server url". The main content area is divided into two sections: "Prontuarios" (Historico clinico dos pets) and "Pets" (Cadastro de pets). The "Prontuarios" section lists three GET endpoints: "/api/v1/prontuarios/{id}" (Buscar prontuario por ID), "/api/v1/prontuarios/pet/{petId}" (Historico clinico de um pet), and "/api/v1/prontuarios/meus" (Prontuarios que eu registrei). The "Pets" section lists five endpoints: a DELETE endpoint "/api/v1/pets/{id}" (Remover pet), and four GET endpoints: "/api/v1/pets/{id}" (Buscar pet por ID), "/api/v1/pets" (Listar pets da clinica), "/api/v1/pets/meus" (Meus pets), and a POST endpoint "/api/v1/pets" (Cadastrar pet).

4. CRUD completo — validado localmente com SELECT no banco

Operações de Create, Read, Update e Delete no recurso **Pet**, relacionado à tabela **Responsavel** (chave estrangeira `pet . id_resp`). Cada resposta da API é seguida de um **SELECT** direto no PostgreSQL para provar a persistência real (não é dado mockado em memória).

CREATE — POST `/api/v1/pets`

```
{
  "id": 4,
  "nome": "Bolinha",
  "descricao": "Cadastrado na demonstracao local",
  "raca": "Vira-lata",
  "especie": "CAO",
  "dataNascimento": "2024-02-10",
  "responsavelId": 3,
  "nomeResponsavel": "Maria Silva"
}
```

UPDATE — PUT `/api/v1/pets/4`

```
{
  "id": 4,
  "nome": "Bolinha Castrado",
  "descricao": "Atualizado na demonstracao local",
  "raca": "Vira-lata",
  "especie": "CAO",
  "dataNascimento": "2024-02-10"
}
```

SELECT direto no PostgreSQL (confirma INSERT + UPDATE persistidos)

id_pet	nome	raca	id_resp
1	Thor	Golden Retriever	3
2	Nina	Siames	3
3	Bud	Poodle	4
4	Bolinha Castrado	Vira-lata	3

(4 rows)

DELETE — DELETE `/api/v1/pets/4` → HTTP 204

SELECT após o DELETE (linha 4 desaparece, dados originais intactos)

id_pet	nome	raca	id_resp
1	Thor	Golden Retriever	3
2	Nina	Siames	3
3	Bud	Poodle	4

(3 rows)

5. Deploy na Azure (ACR + ACI)

Todos os recursos foram criados via Azure CLI (scripts em `deploy/azure/`):

```
$ az resource list --resource-group rg-rm562917-devops -o table
```

Name	Location	Type	Status
2tdsarm562917	eastus	ContainerRegistry/registries	Succeeded
ollipet-aci	eastus	ContainerInstance/containerGroups	Succeeded

Status do container group (App + Banco no mesmo grupo, IP público):

```
{
  "status": "Running",
  "fqdn": "ollipet-rm562917.eastus.azurecontainer.io",
  "ip": "4.157.234.69",
  "containers": [
    { "nome": "db", "estado": "Running", "restarts": 0 },
    { "nome": "app", "estado": "Running", "restarts": 1 }
  ]
}
```

Swagger UI publicado (repare a URL do servidor: FQDN da Azure, não localhost):

The screenshot shows the Swagger UI interface for an OpenAPI definition. At the top, there's a Swagger logo and a search bar containing "/api-docs" with an "Explore" button. Below this, the title "OpenAPI definition" is displayed with a "v0" tag and "OAS 3.1" specification version. A "Servers" section shows a dropdown menu with the selected server URL: "http://ollipet-rm562917.eastus.azurecontainer.io:8080 - Generated server url". The main content area is divided into two sections: "Prontuários" (Historico clinico dos pets) and "Pets" (Cadastro de pets). The "Prontuários" section lists three GET endpoints: "/api/v1/prontuarios/{id}" (Buscar prontuario por ID), "/api/v1/prontuarios/pet/{petId}" (Historico clinico de um pet), and "/api/v1/prontuarios/meus" (Prontuarios que eu registrei). The "Pets" section lists five endpoints: a DELETE endpoint "/api/v1/pets/{id}" (Remover pet), and four GET endpoints: "/api/v1/pets/{id}" (Buscar pet por ID), "/api/v1/pets" (Listar pets da clinica), "/api/v1/pets/meus" (Meus pets), and a POST endpoint "/api/v1/pets" (Cadastrar pet).

6. CRUD completo — validado direto na API pública da Azure

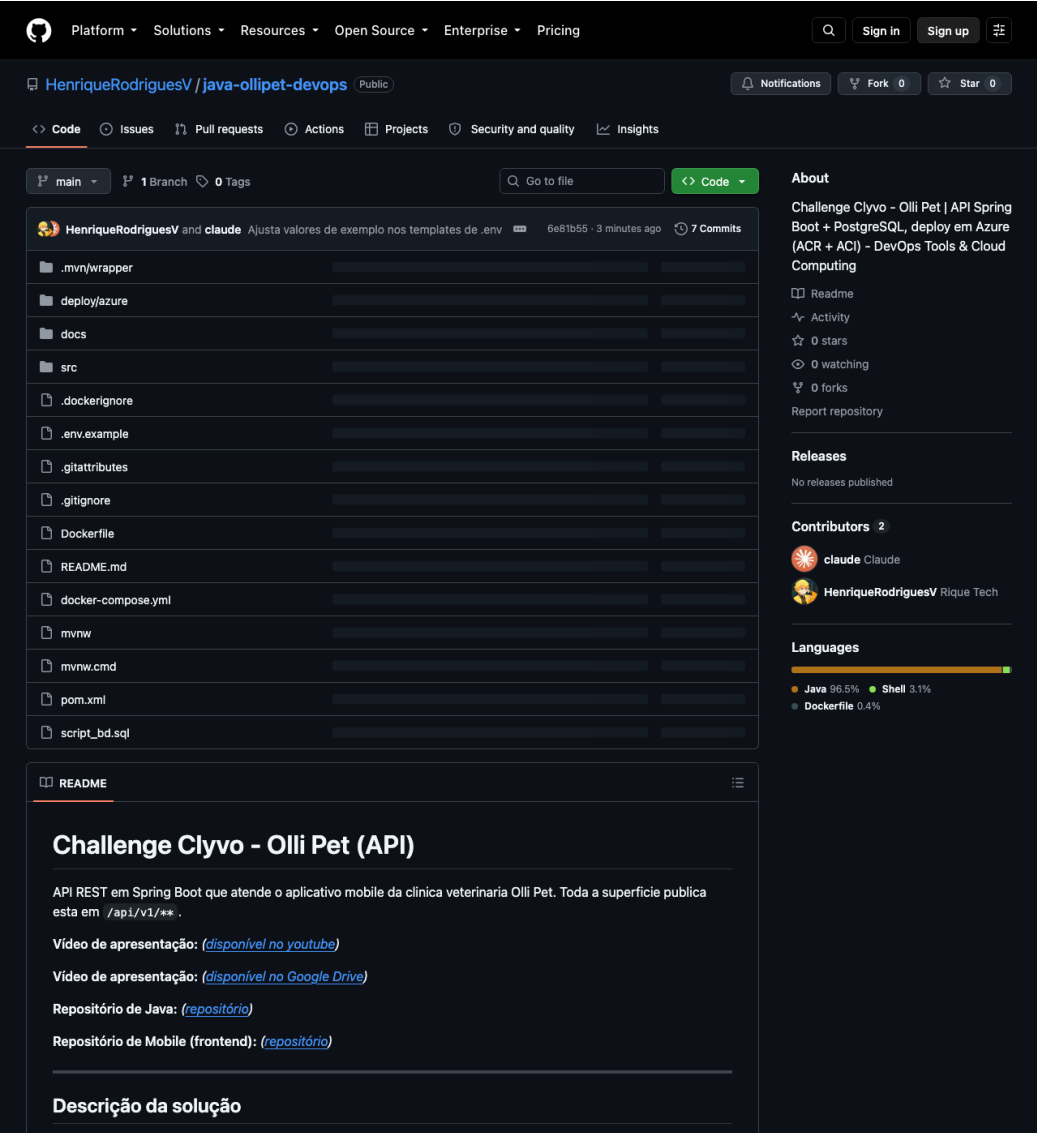
Mesmo fluxo do item 4, agora contra o ambiente publicado (não localhost):

```
CREATE -> HTTP 200 { "id": 5, "nome": "Nuvem", "raca": "SRD", ... }
READ   -> HTTP 200 { "id": 5, "nome": "Nuvem", ... }
UPDATE -> HTTP 200 { "id": 5, "nome": "Nuvem Atualizado", ... }
DELETE -> HTTP 204
READ (apos DELETE) -> HTTP 404 (confirma remocao)
```

Para a gravação do vídeo, o SELECT direto no banco em nuvem deve ser feito com um shell interativo dentro do container (evita um bug de parsing de aspas do `az container exec` em modo não-interativo) — passo a passo completo em `docs/roteiro-video-sprint3.md`.

7. Código-fonte no GitHub

Repositório público, com histórico de commits real (não é upload único):



The screenshot displays the GitHub interface for the repository `HenriqueRodriguesV/java-ollipet-devops`. The repository is public and has 1 branch and 0 tags. The file list includes `.mvn/wrapper`, `deploy/azure`, `docs`, `src`, `.dockerignore`, `.env.example`, `.gitattributes`, `.gitignore`, `Dockerfile`, `README.md`, `docker-compose.yml`, `mvnw`, `mvnw.cmd`, `pom.xml`, and `script_bd.sql`. The `README` file is selected and shows the following content:

Challenge Clyvo - Olli Pet (API)

API REST em Spring Boot que atende o aplicativo mobile da clinica veterinaria Olli Pet. Toda a superficie publica esta em `/api/v1/**`.

Vídeo de apresentação: ([disponível no youtube](#))

Vídeo de apresentação: ([disponível no Google Drive](#))

Repositório de Java: ([repositório](#))

Repositório de Mobile (frontend): ([repositório](#))

Descrição da solução

The right sidebar shows repository statistics: 0 stars, 0 forks, and 0 watchers. It also lists contributors: `claudio` and `HenriqueRodriguesV`. The languages section shows Java at 96.5%, Shell at 3.1%, and Dockerfile at 0.4%.

<https://github.com/HenriqueRodriguesV/java-ollipet-devops>

8. Estrutura do banco de dados

DDL completo e comentado (tabelas do CORE: usuario, responsavel, med_vet, pet, prontuario, consulta, vacina, aplicacao_vacina, triagem e tratamento) está em `script_bd.sql`, na raiz do repositório.

9. Link do vídeo

(cole aqui o link do YouTube apos gravar)